Инструменты промышленной разработки

Семинар 2 · Пакетные менеджеры · Open source и лицензии · Markdown, Mermaid и Marp · LaTeX

Семинарист: Шевердов Никита

Что внутри

1 Пакетные менеджеры: зачем они нужны, что внутри пакета и почему это NP-полная задача

2 Open source и лицензии: MIT, Apache, GPL, Creative Commons, WTFPL и «код без лицензии»

3 Markdown и его экосистема: шпаргалка синтаксиса, Mermaid (схемы как код), Marp (слайды)

4 LaTeX: как устроен документ, математика, ссылки на формулы, Overleaf

5 Прикладные советы: hse-tex и выводы семинара

Проверяемые факты снабжены источниками в разделе «Источники»

1. Пакетные менеджеры

Задача, из которой всё выросло, простая: написать библиотеку или утилиту, упаковать её в пакет и передать другим — так, чтобы пакет ставился целиком, поддерживал версионирование и не подменялся «фейковой» версией. Первые пакетные менеджеры появились в 80-х; сегодня они есть у каждой операционной системы и у каждого языка программирования.

- apt — Debian/Ubuntu; Homebrew (brew) — macOS (работает и на Linux); pacman — Arch Linux; pip — менеджер библиотек Python.
- Ту же идею развивают языковые менеджеры: npm (JavaScript), Cargo (Rust) и другие.
- Пакет — не только код. Внутри лежат метаданные: версия, зависимости, лицензия, ссылка на проект — всё это читается менеджером при установке.

Заглянуть в кишки установленного Python

В интерпретаторе Python sys.modules — словарь всех уже загруженных модулей. После import fractions в его конце появляется запись о новом модуле — так видно, что «вошло» в питон. Файлы библиотек лежат в site-packages, а внутри каждого пакета — файл METADATA с версией, homepage и лицензией.

```
$ pip show aiohttp                # версия, лицензия, homepage, зависимости
Name: aiohttp
Version: 3.x.x
License: Apache-2.0
Requires: attrs, yarl, ...

$ python3 -c "import aiohttp, pathlib; print(pathlib.Path(aiohttp.__file__).parent)"
.../site-packages/aiohttp           # сюда ставятся все библиотеки
```

sys.modules — это словарь

sys.modules — словарь (имя модуля → сам модуль), отражающий то, что питон загрузил в память за текущую сессию. Появление записи в нём — эффект импорта, а не состояние «установленных пакетов»: список установленных смотрят через pip list / pip show.

Самое сложное — зависимости

Пакет может зависеть от других пакетов, те — от следующих. Отсюда две классические боли: циклические зависимости (A зависит от B, B от C, C от A) и конфликты версий: одной библиотеке нужен пакет B версии 2.0 и выше, а другой — 1.9 и ниже. Менеджеры решают это по-разному: apt держит одну версию пакета в репозитории и старается сводить конфликты «по-умному»; у Homebrew есть версионные формулы вида node@24 (их можно ставить рядом с основной версией); языковые менеджеры вроде pip собирают согласованный набор версий на одно окружение. Задача «выбрать набор версий, удовлетворяющий всем ограничениям» — NP-полная.

```
# классический конфликт версий
requirements.txt (проект A):  numpy>=2.0
requirements.txt (проект B):  numpy<2.0
# одно системное окружение не удовлетворит обоим ->
# нужны изолированные окружения (venv, conda) и lock-файлы
```

Чем строже ограничения — тем легче резолверу

Свободные требования вида numpy>=0 или * разворачивают дерево перебора и плодят дублирующиеся версии. Пиньте версии, коммитьте lock-файлы — и установка станет быстрой и воспроизводимой. Полюбить пакетные менеджеры придётся в любом случае: с приходом Docker и виртуальных машин конфликты версий встретятся в полный рост.

2. Open source и лицензии

Откуда это всё взялось

Классическая точка отсчёта — Лаборатория искусственного интеллекта MIT начала 80-х: внутри неё десятилетиями жила культура свободного обмена кодом, и именно из неё выросли GNU, свободное ПО и современный open source.

> **История: от MIT AI Lab к GNU**
>
> В 1979–1980 из MIT AI Lab выделились две коммерческие компании — Lisp Machines Inc. и Symbolics (коммерциализация Lisp-машин). Symbolics забрал большую часть сотрудников и стал закрывать доработки общего софта. Ричард Столлман (автор Emacs) остался в лаборатории и около двух лет (1982–1983) вручную переписывал новые возможности Symbolics обратно в свободную версию. В 1983 он объявил проект GNU, затем появились Free Software Foundation и лицензия GPL. Итого: open source — не результат спонтанной «щедрости», а ответ на коммерциализацию, разрушившую культуру свободного обмена кодом.

Зачем open source студенту

- GitHub — твоё резюме. Профиль с живыми контрибьюциями читают чаще, чем PDF.
- Реальный пример: инженер устроился в команду ClickHouse после того, как ещё на третьем курсе много и добровольно писал туда код.
- Коммить в open source — тренировка чтения чужого кода и прохождения ревью.
- Платформы: GitHub и GitLab — для кода, Hugging Face — для ML-артефактов и моделей.
- Багбаунти: корпорации платят за найденные уязвимости в чужом коде. А вот за разбор чужих лицензий денег не платят: это просто гигиена любого проекта.

Лицензия — главный юридический узел

Open Source Initiative (OSI) — организация, проверяющая лицензии на соответствие определению open source. Код — интеллектуальная собственность, и лицензия отвечает ровно на один вопрос: что с ним можно делать. Сводка основных вариантов:

Лицензия	Суть	Обязательства	Когда выбирать
MIT	Почти всё разрешено	Сохранить копирайт и текст лицензии	Дефолт для своих библиотек и пет-проектов
Apache 2.0	Как MIT + явный патентный грант	То же + помечать изменения	Корпоративные проекты, важны патенты
GPL-3.0	Копилефт («вирусная»)	Производное — только под GPL	Когда проект должен остаться свободным
CC BY / BY-SA / NC / ND	Семейство Creative Commons	BY — указать автора; SA — та же лицензия; NC — не для коммерции; ND — без изменений	Тексты, картинки, данные; для кода обычно не берут
CC0	Отказ от прав (public domain)	Ничего	Отдать материал без условий

Лицензия	Суть	Обязательства	Когда выбирать
WTFPL	«Делай что хочешь»	Фактически ничего	Не рекомендуется: не одобрена OSI, нет патентного гранта
Нет лицензии	«Все права защищены»	—	Нельзя использовать без разрешения автора

Таблица 1. Основные лицензии в одном взгляде

Creative Commons

Creative Commons (CC) — семейство лицензий для творческих произведений: шесть основных комбинаций из элементов BY (атрибуция), SA (та же лицензия), NC (некоммерческое), ND (без изменений) плюс CC0. Важно: CC придуманы для текстов, фотографий и данных, а не для кода — для кода обычно берут MIT/Apache/GPL.

WTFPL

WTFPL (DO WHAT THE FUCK YOU WANT TO PUBLIC LICENSE) — короткая лицензия: вторая версия (декабрь 2004, Sam Hocevar) сводится к «0. You just DO WHAT THE FUCK YOU WANT TO». Минусы: не одобрена OSI, нет патентного гранта, несовместима с GPL v2 — поэтому серьёзные проекты обычно выбирают MIT или 0BSD.

Код без LICENSE

Репозиторий без файла LICENSE — это код под «все права защищены»: читать можно, вставлять в свой проект — нельзя. Лицензию надо добавлять сознательно.

curl и атрибуция в титрах игр

У curl — MIT-производная лицензия (curl license): использовать, менять и даже встраивать в проприетарные продукты можно, но копирайт-уведомление должно сохраняться в копиях. Команда curl ведёт список проектов, где используется libcurl, — от операционных систем до игр: атрибуция реально встречается в титрах (в том числе в GTA V).

Практический риск: в кодовую базу компании попадает библиотека под жёсткой лицензией — и продукт, который нельзя продавать. Плюс классика: сотрудник выносит кусок кода наружу и «судит собственную компанию». Отсюда простое правило: перед тем как тянуть зависимость в продукт, читай её лицензию (позже это называется license audit). Лицензии учитывают и российские суды — хотя вся юридика в первую очередь писалась под США.

3. Markdown и его экосистема

Markdown — облегчённая разметка для обычного текста: документ читается даже без рендеринга и конвертируется в HTML. Файлы с расширением .md — это и README на GitHub, и документация компаний, и заметки.

Историческая справка: язык создал Джон Грубер при участии Аарона Шварца в 2004 году; многие идеи взяты из разметки электронных писем. За последние годы markdown из «нишевой мелочи» стал стандартом де-факто: README на GitHub, Яндекс Вики и Документация, Notion, Obsidian, Telegram и Discord, ответы нейросетей. Markdown не «полностью заменяет HTML»: это тонкая надстройка над HTML, в которую при необходимости можно вставлять сам HTML.

Шпаргалка синтаксиса

Синтаксис	Результат
`# Заголовок ## Заголовок 2`	Заголовок 1-го и 2-го уровня
`**жирный**  *курсив*  ~~зачёркнутый~~`	Форматирование текста
`- пункт (или * пункт)`	Маркированный список
`1. пункт (нумерация часто не важна)`	Нумерованный список
`[текст](https://example.com)`	Ссылка
`![alt](путь/картинка.png)`	Картинка (восклицательный знак!)
`` `код` и ```язык ... ``` ``	Код инлайн и блоком с подсветкой
`> цитата`	Цитата
`| a | b | --- | --- |`	Таблица
`- [ ] задача  - [x] сделано`	Чек-листы
`---`	Горизонтальная линия (и разделитель слайдов в Marp)
`два перевода строки`	Новый абзац (одинарный перевод — не абзац!)

Таблица 2. Всё, что нужно для заметок и простых отчётов

Mermaid: диаграммы как код

Логичное продолжение мысли «всё, что можно хранить в тексте, должно жить в тексте». Схема архитектуры (какой сервис в какой сервис ходит, где база) нужна системным аналитикам постоянно, но рисовать её мышкой в клик-инструментах плохо по двум причинам: у картинки нет версионирования, а два параллельных изменения такой «схемы» почти невозможно смержить без конфликта. Кодом — обычный diff и merge request.

Mermaid

Mermaid — отдельный инструмент «диаграммы как код»: текстовый язык, который рендерится JS-библиотекой. Синтаксис markdown-подобный, но это не markdown. Автор проекта — Кнут Свейдквист (Knut Sveidqvist). В проекте 20+ типов диаграмм: flowchart, sequence, class, state, ER, Gantt, mindmap, git graph и другие. GitHub рендерит Mermaid нативно с февраля 2022; поддерживают её GitLab, Notion, Obsidian; есть онлайн-редактор mermaid.live.

````
```mermaid
flowchart LR
    Клиент --> API-сервер --> База данных[(PostgreSQL)]
```

```mermaid
sequenceDiagram
    Alice->>John: Привет!
    John-->>Alice: Привет!
```
````

Выше — исходный код, ниже — то, как Mermaid его отрисует:



Marp: презентации из markdown

Marp (Markdown Presentation Ecosystem) превращает markdown-файл в слайды. Слайды разделяются строкой из трёх дефисов --- (как горизонтальная линейка в markdown). Достаточно добавить front-matter marp: true — и расширение Marp for VS Code покажет слайды рядом с кодом, а консольный marp CLI экспортирует их в HTML, PDF и даже PPTX.

```
---
marp: true
theme: default
paginate: true
---

# Заголовок первого слайда

- пункт 1
- пункт 2

---

## Второй слайд

Код, картинки, формулы — всё как в обычном markdown
```

LLM и слайды

Нейросети заметно охотнее пишут презентации в markdown: вместо кучи HTML им достаточно текста с разделителями — и тебе править проще. Для строгого дизайна всё равно придётся допиливать CSS-темы, но черновик в Marp собирается за минуты.

Где markdown не тянет: книги и курсовые (нет контроля вёрстки, сносок, авто-нумерации высокого уровня) и серьёзная математика (формулы зависят от рендерера). Это прямой переход к следующей теме — LaTeX.

4. LaTeX

Историческая справка

TeX создал Дональд Кнут: первая версия (TeX78) вышла в 1978 году, позже система переписана заново как TeX82. Причина — Кнута не устроило качество вёрстки второго издания его «Искусства программирования». Кнут же придумал язык описания шрифтов Metafont и концепцию literate programming. LaTeX — макропакет над TeX, который написал Лесли Лэмпорт в 1980-е (его книга о LaTeX вышла в 1986-м); современная версия LaTeX2e развивается с 1994 года. В 90-х LaTeX окончательно стал стандартом академической вёрстки.

Философия: WYSIWYM

В Word ты управляешь внешним видом (что видишь — то и получишь, WYSIWYG), и красивую вёрстку там надо уметь делать руками: поехали переносы, текст наехал на текст — и полстраницы ремонта. В LaTeX ты описываешь структуру и смысл (это раздел, это формула, это список), а переносы, отступы, нумерацию и красоту делает компилятор (WYSIWYM — what you see is what you mean). После пары часов практики о «красоте» вообще перестаёшь думать. Поэтому научные статьи, курсовые, дипломы и резюме в IT — почти всегда LaTeX.

```
\documentclass{article}
\usepackage[russian]{babel}
\usepackage{amsmath}

\title{Отчёт по курсовой работе}
\author{И.\ Иванов}
\date{\today}

\begin{document}
\maketitle
\section{Введение}
Первый абзац. Ещё одна строка
того же абзаца.

Пустая строка выше = новый абзац.
\section{Метод}
Среднее значение задаётся формулой~\eqref{eq:mean}:
\begin{equation}
  \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i
  \label{eq:mean}
\end{equation}
\end{document}
```

- Пустая строка = новый абзац. Перенос строки без пустой — тот же абзац: длинные строки удобно «причёсывать» в редакторе, не ломая текст.
- Списки: `itemize` — маркированный, `enumerate` — нумерованный. Номера проставляются сами — как в markdown.
- Математика: внутристрочно — в $...$, вынесенная — в \[...\] или окружении equation. Дроби \frac, суммы \sum, черта \bar, индексы x_i.
- Ссылки по имени, а не по номеру: \label ставит метку, \ref/\eqref/\cite ссылаются на неё; пакет hyperref делает PDF кликабельным. При перестановке разделов номера пересчитаются сами — вручную ничего править не надо.
- Вспомогательные файлы .aux/.toc — данные между проходами компиляции: номера страниц и ссылки определяются за два прохода, поэтому после первого запуска ссылки

могут показываться как ?? — просто скомпилируй ещё раз.

- Картинки — исторически самое муторное место TeX. Но пакеты TikZ и pgfplots позволяют рисовать векторную графику и даже 3D-сцены прямо кодом.

Формула: источник и результат

Исходник: `\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i`. После компиляции — читаемая формула:

$$\bar{x} = (1/n) \sum_{i=1}^{n} x_i \quad \text{(среднее арифметическое)}$$

Где и как работать

- Overleaf — облачный редактор и компилятор; базовая функциональность бесплатна, проблемы начинаются с коворкингом и очень большими проектами.
- Локально: TeX Live (Linux/macOS) или MiKTeX (Windows), компиляция командой pdflatex/xelatex — PDF собирается за секунды даже для больших работ.
- Документ можно разбить на несколько файлов и включать их друг в друга, как модули в коде.
- Резюме в LaTeX — множество открытых шаблонов: выглядит профессионально и легко парсится автоматикой.

Прежде чем писать что-то своё для LaTeX

Для LaTeX написано огромное количество библиотек и модулей. Если захотелось что-то «доработать» — сначала погугли: почти наверняка нужный пакет уже существует (стрелочки, матрицы, диаграммы, таблицы). Свой макрос — только если готового решения нет.

5. Прикладные советы

Учёба

- hse-tex — открытый репозиторий затеханных конспектов, коллоквиумов и экзаменов ФКН ВШЭ (сайт hse-tex.me, Telegram @hsetex). Большинство материалов не сильно устарели: математика за это время не ушла далеко. Опечатку нашёл — исправь в PR.

Три главных вывода

1	Используйте пакетные менеджеры осознанно: боли с версиями неизбежны (особенно с Docker и виртуальными машинами), но lock-файлы и фиксация версий делают жизнь предсказуемой.
2	Коммитьте в open source: это тренировка, портфолио и способ сделать мир лучше — мир может ответить и на карьерный рост.
3	Word → Markdown и LaTeX: всё, что можно хранить в тексте, должно жить в тексте — так оно версионируется, мержится и не теряется.

Источники

· research.swtch.com/version-sat
NP-полнота задачи выбора версий пакетов (Version SAT, Russ Cox, 2016)

· habr.com/ru/articles/652867
нативная поддержка Mermaid в GitHub с 14.02.2022; типы диаграмм

· github.com/mermaid-js/mermaid
Mermaid: «diagrams as code», 20+ типов диаграмм, автор Knut Sveidqvist

· marketplace.visualstudio.com/items?itemName=marp-team.marp-vscode
Marp: front-matter marp: true, экспорт HTML/PDF/PPTX

· github.com/curl/everything-curl — opensource/license.md
curl license — MIT-производная, обязательное копирайт-уведомление

· en.wikipedia.org/wiki/CURL
curl license вдохновлена MIT; curl используется повсеместно

· ru.wikipedia.org/wiki/Markdown
Markdown: Джон Грубер и Аарон Шварц, 19 марта 2004

· opensource.stackexchange.com/questions/13806
текст WTFPL v2 (декабрь 2004, Sam Hocevar); WTFPL не одобрена OSI

· skillbox.ru/media/code/kak-microsoft-borolas-s-open-source
GNU Ричарда Столлмана (1983), GPL и копилефт

· habr.com/ru/companies/yandex/articles/519922
«война» Столлмана и Symbolics (1982–1983), Lisp Machines

· old.computerra.ru/201421
TeX Кнута, LaTeX Лэмпорта — появился в 1980-е

· dev.to/serverspace/leslie-lamport-the-man-who-invented-latex-m9e
Лесли Лэмпорт: LaTeX как набор макросов над TeX, книга 1986

· cartetika.ru/tpost/f0a7986hc1-litsenzii-creative-commons
шесть основных лицензий Creative Commons + CC0

- hse-tex.me
  hse-tex: затеханные конспекты и коллоквиумы ПМИ ФКН ВШЭ
- heatware.net/macos/brew-install-specific-version
  версионные формулы Homebrew вида package@X

Конспект семинара 2 курса «Инструменты промышленной разработки» (семинарист — Шевердов Никита). Факты сверены с источниками выше.